

boltzmann

October 4, 2026

Facultad de Ingeniería Mecánica y Eléctrica

Ingeniería en Computación Inteligente

7mo B

Integrantes del equipo:

- *Christian David Sánchez Sánchez*
- *Marcela Betzabeth Márquez González*

Fecha de entrega: 5 de octubre de 2026

0.1 Introducción

Una máquina de Boltzmann es un tipo específico de red neuronal estocástica cuya estructura incluye unidades de procesamiento interconectadas que operan de manera probabilística. A diferencia de otras redes neuronales, las máquinas de Boltzmann son especialmente útiles para modelar y simular sistemas complejos y dinámicos debido a que pueden capturar la variabilidad e incertidumbre inherentes en muchos problemas del mundo real [1]. Una de sus características principales es que se considera una red neuronal *recurrente*, las cuales se distinguen por su “memoria” ya que toman información de entradas anteriores para influir en la entrada y salida actual [2].

¿Cómo puede una red neuronal estocástica aprender una distribución de probabilidad y cuáles son las dificultades que limitan su entrenamiento? Para poder responder a esta pregunta, es necesario primero entender los conceptos básicos que nos llevan a cuestionarla. Un *proceso estocástico* es todo proceso que describe la evolución de un sistema a lo largo del tiempo donde el azar tiene una intervención. A diferencia de los modelos deterministas, donde el resultado está totalmente definido por las condiciones iniciales, en estos procesos hay incertidumbre en los resultados [3].

De este modo, podemos definir a las *redes neuronales estocásticas* como modelos caracterizados por las variaciones aleatorias ya sea en sus neuronas o en sus pesos. Estas resultan especialmente útiles para resolver problemas de optimización ya que, gracias a estas fluctuaciones, podemos crear algoritmos que eviten los mínimos locales además de modelar y simular sistemas complejos.

Su principal limitación son los tiempos de entrenamiento, ya que es extremadamente lento y computacionalmente caro debido a la necesidad de calcular el equilibrio térmico mediante métodos de muestreo avanzados.

0.2 Antecedentes

En 1982, John Hopfield introdujo la *Red de Hopfield*, una red neuronal recurrente de unidades binarias con conexiones simétricas que funcionaba como un sistema dinámico con memoria asociativa. Su funcionamiento es simple: dada una configuración inicial, la red actualiza asincrónicamente el estado de sus nodos hasta lograr la convergencia a un estado estable. Este modelo, a pesar de haber revolucionado el área de la inteligencia artificial, su dinámica descendente provocaba que la red se quedara bloqueada en configuraciones no óptimas sin poder alcanzar un mínimo global [4].

Para resolver las limitaciones en sistemas energéticos, se desarrolló la técnica de *Recocido Simulado*, el cual introduce ruido estocástico controlado en las reglas de transición entre estados para que el sistema adquiera la capacidad de realizar saltos probabilísticos hacia estados de mayor energía. Cuando se obtienen temperaturas elevadas, el sistema explora el espacio disponible de estados; a medida que la temperatura disminuye, su probabilidad de visitar estados desfavorables también [5].

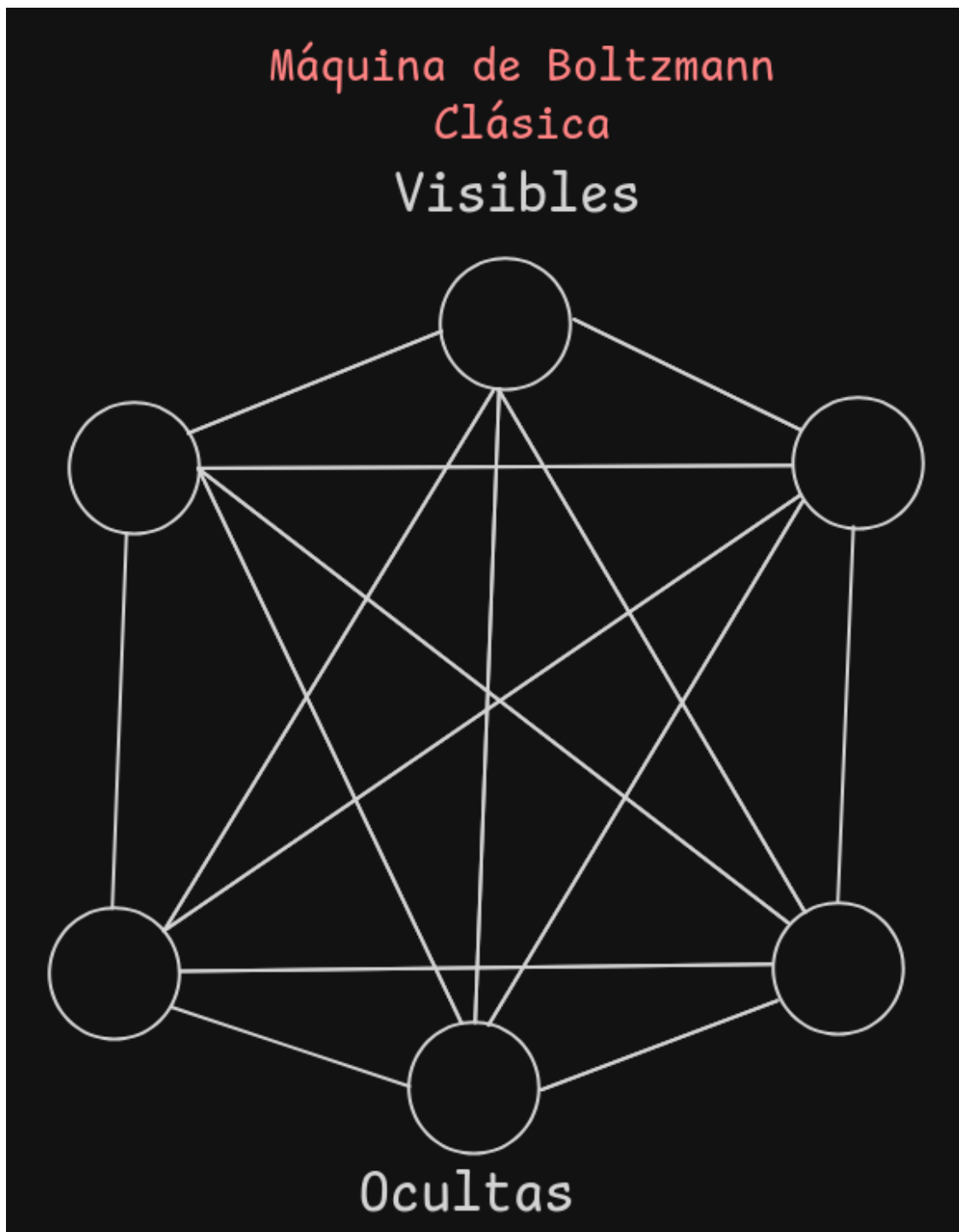
En 1985 se consolidó la *máquina clásica de Boltzmann*, unificando la estructura de la *red de Hopfield* y la técnica del *Recocido Simulado*. Entre sus tantas innovaciones y contribuciones teóricas están [2]:

- Se dividieron las neuronas en unidades visibles (las cuales reciben la entrada/salida de datos) y unidades ocultas (las cuales capturan correlaciones complejas).
- Se reemplazó la función de activación determinista por reglas de transición probabilística.
- Se demostró que, al correr la red libremente a una temperatura fija durante el tiempo suficiente, la probabilidad de observar cualquier estado global converge a la distribución estocástica de Boltzmann.

A pesar de tener un sustento y solidez matemática, este modelo resulta computacionalmente inviable para aplicaciones prácticas debido al intenso costo computacional que representa utilizarlo. Se propusieron algunos modelos derivados que superan este obstáculo, como la *Máquina de Boltzmann Restringida* o la *Restricción Bipartita* [6].

0.3 Arquitectura

La arquitectura de una máquina de Boltzmann se puede describir con el siguiente diagrama:



En donde:

- **Unidades visibles:** Representan las variables observables del entorno (datos de entrada)

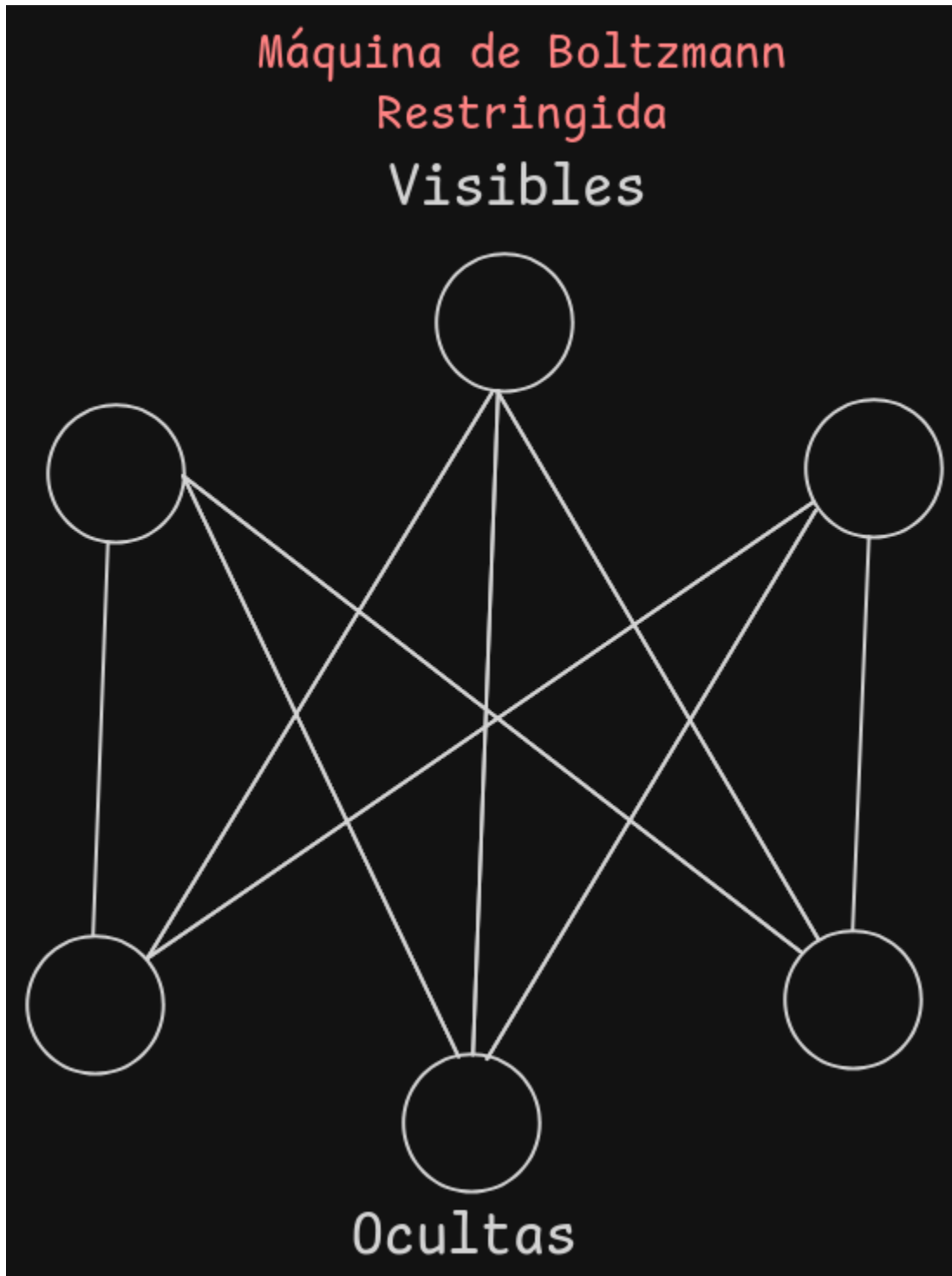
[7].

- **Unidades ocultas:** Actúan como detectores de características no lineales que capturan correlaciones complejas entre variables visibles [7].
- **Pesos (W):** Representa el peso de la conexión simétrica entre la unidad visible y la unidad oculta, es decir, que tan importante será esa entrada en la toma de decisiones [7].
- **Sesgos:** Cada unidad visible/oculta tiene un valor de sesgo, el cual accion a cada unidad para que ejecute una acción y nunca quede sin hacer nada (muerta) [7].

Cada una de estas unidades tiene dos estados posibles: 0 o 1. Es decir, sus estados son totalmente binarios.

Como podemos notar en esta arquitectura, la clave es que cada una de las neuronas está conectada entre si, incluso las de la misma capa, lo que hace que cada unidad sea dependiente de otras las cuales provienen de multiples capas.

En cambio, la arquitectura de la máquina de Boltzmann restringida es:



Podemos notar que la principal diferencia entre ambas es que esta (RBM) prohíbe explícitamente las conexiones internas dentro de la misma capa. Esto garantiza que, dado el estado de la capa visible todas las unidades ocultas y visibles son condicionalmente independientes entre si por lo que se permite realizar inferencia probabilística [8].

0.4 Comportamiento probabilístico

El estado de las unidades en arquitecturas deterministas (como la red de Hopfield o perceptrón) se rige por la función escalón o umbral, la cual indica que si una neurona se activa basandose en el valor de la suma ponderada. El uso de esta función es bastante simple y entendible, pero tiene un problema en modelos basados en energía: no permite transicionar a estados de mayor energía, lo que ocasiona que pueda quedar atrapado en mínimos locales. Además, elimina el factor estocástico el cual es necesario para permitir al modelo atravesar barreras energéticas, escapar de mínimos locales o explorar el espacio de estados a su conveniencia [].

La expresión utilizada es la siguiente:

$$P(s_i = 1) = \frac{1}{1 + e^{-\frac{\Delta E_i}{T}}}$$

Donde:

- $P(s_i = 1)$: Representa la probabilidad de que la neurona cambie o se mantenga en el estado activo, dado el estado del resto de las neuronas presentes en la red.
- s_i : Variable estocástica binaria que representa su estado actual (0 o 1).
- ΔE_i : Representa la diferencia de energía o la entrada neta, siendo la variación global que experimenta la red cuando alguna neurona se activa.
- T : Representa el nivel de ruido estocástico o fluctuación térmica del sistema. En caso de que $T = 0$, esta actuará como una función escalón convencional, conforme aumenta su valor el ruido que introduce también lo hace y cuando disminuye el ruido que introduce también lo hace.
- e : Constante de Euler, la cual es la base de los logaritmos naturales.
- $\frac{1}{1 + e^{-\frac{\Delta E_i}{T}}}$: Representa la función logística la cual toma cualquier valor de entrada y lo mapea a un rango continuo entre 0 y 1, lo que nos permite interpretarlo como una probabilidad.

Al realizar dos ejecuciones, el sistema produce dos secuencias de estados diferentes debido a la naturaleza estocástica inherente al modelo y a su mecanismo de transiciones probabilísticas. El estado global de la red (ΔE_i) se actualiza paso a paso mediante el *Muestreo de Gibbs*, realizando un proceso de “cadena” donde cada estado x_{i+1} se muestrea probabilísticamente con todos sus estados anteriores.

0.5 Función de energía

Para definir estados binarios como energía, se utiliza la siguiente ecuación:

$$E(s) = -\frac{1}{2} \sum_i \sum_j w_{ij} s_i s_j - \sum_i b_i s_i$$

Esta ecuación define como calcular la energía global del estado ($E(s)$) basado en los siguientes parámetros:

- $E(s)$: Valor escalar que mide el nivel de conflicto o desacuerdo en la red para una configuración particular de neuronas s . A menor valor de energía, menor será la probabilidad de que el sistema visite o genere dicho estado.
- w_{ij} : Representa la intensidad de la conexión e interacción entre la neurona i y la neurona j . Entre mayor sea el valor, la red premiará la activación simultanea de ambas neuronas por lo que la energía disminuirá.
- s_i, s_j : Variables estocásticas y binarias de las neuronas i y j . El valor de su producto es máximo 1.
- Factor $\frac{1}{2}$: Al realizar una doble suma sobre todos los índices i y j , cada par de neuronas (i, j) y (j, i) se contabiliza dos veces, por lo que se debe corregir para que solo contribuya una sola vez a la energía total del sistema.
- b_i : Parámetro que determina la preferencia inherente de la neurona i a estar activa, independientemente del estado actual de las demás neuronas.
- s_i (Segundo termino): Representa el estado de la neurona i , multiplicando al sesgo b_i , lo que garantiza que el sesgo solo altere la energía global cuando la neurona esté encendida.

Ahora, ¿por qué el signo negativo? Este garantiza que las configuraciones deseables reduzcan la energía global y, por ende, tengan mayor probabilidad de ocurrencia.

Realizaremos el cálculo de 3 estados. Para efectos de la práctica, se utilizaron los siguientes valores:

- $S_1 = (1, 1, 0)$
- $S_2 = (1, 0, 1)$
- $S_3 = (0, 0, 0)$
- $W_{12} = 2$
- $W_{13} = -1.5$
- $W_{23} = 1$
- $b_1 = 0.5$
- $b_2 = -1$
- $b_3 = 0$

$w_{12} = 2$	$w_{13} = -1.5$	$w_{23} = 1$	$b_1 = 0.5$	$b_2 = -1$	$b_3 = 0$
Estado 1: $s_1 = (1, 1, 0)$	Estado 2: $s_2 = (1, 0, 1)$	Estado 3: $s_3 = (0, 0, 0)$			
$w_{12} s_1 s_2 = 2 \cdot 1 \cdot 1 = 2$	$w_{12} s_1 s_2 = 2 \cdot 1 \cdot 0 = 0$	$w_{12} s_1 s_3 = 2 \cdot 0 \cdot 0 = 0$			
$w_{13} s_1 s_3 = -1.5 \cdot 1 \cdot 0 = 0$	$w_{13} s_1 s_3 = -1.5 \cdot 1 \cdot 1 = -1.5$	$w_{13} s_1 s_3 = -1.5 \cdot 0 \cdot 0 = 0$			
$w_{23} s_2 s_3 = 1 \cdot 1 \cdot 0 = 0$	$w_{23} s_2 s_3 = 1 \cdot 0 \cdot 1 = 0$	$w_{23} s_2 s_3 = 1 \cdot 0 \cdot 0 = 0$			
Suma total = 2	Suma total = -1.5	Suma total = 0			
Sesgos:	Sesgos	Sesgos			
$0.5 \cdot 1 + (-1)(1) + (0)(0)$	$(0.5)(1) + (-1)(0) + (0)(1)$	$(0.5)(0) + (-1)(0) + (0)(0)$			
$= 0.5 - 1 + 0 = -0.5$	$= 0.5 + 0 = 0.5$	$= 0$			
$E(s_1) = -2 - (-0.5) = -1.5$	$E(s_2) = -(-1.5) - 0.5 = 1$	$E(s_3) = -0 + 0 = 0$			

Podemos notar que el estado 1 es el que resultó con la menor energía, mientras que el estado 2 fue el mayor.

El comportamiento probabilístico general de estas máquinas se basa en la conexión de la energía, estabilidad y probabilidad.

La relación entre la energía y la probabilidad en las máquinas de Boltzmann está formalizada por la distribución de Gibbs-Boltzmann, según la cual la probabilidad conjunta de que el sistema se encuentre en un estado global determinado s disminuye de manera exponencial a medida que aumenta su función de energía $E(s)$. Las configuraciones con menor energía adquieren una mayor probabilidad de ocurrencia al representar estados estables que satisfacen óptimamente las restricciones de la red, mientras que los estados de alta energía son penalizados y resultan poco probables

0.6 Implementación en Python

En esta sección, desarrollaremos la implementación de la máquina de Boltzmann en Python utilizando librerías estandar que nos ayudarán en los calculos matemáticos, más no nos crearán el modelo automáticamente. Se desarrolla con el fin de comprender algorítmicamente el proceso en código del lenguaje estándar de la Inteligencia Artificial.

Para esto, se utilizarán las siguientes librerías:

```
[1]: import pandas as pd # Para la lectura de los datos iniciales
import numpy as np # Para los procesos algebraicos necesarios
```

0.7 Referencias bibliográficas

- [1]. Msmk. (2024, 31 octubre). Máquina de Boltzmann. Top Up Grado En un Año MSMK. <https://msmk.university/maquina-de-boltzmann/>
- [2]. ¿Qué es una red neuronal recurrente (RNN)? | IBM. (2026, 16 agosto). <https://www.ibm.com/mx-es/think/topics/recurrent-neural-networks>
- [3]. Universidad Europea. (2026, 10 junio). Procesos estocásticos: qué tipos existen y cuáles son sus aplicaciones. <https://universidadeuropea.com/blog/procesos-estocasticos/>
- [4]. Ackley, D. H., Hinton, G. E., Sejnowski, T. J. (1985). A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1), 147–169. https://doi.org/10.1207/s15516709cog0901_7
- [5]. Restricted Boltzmann Machines, Bernoulli Mixtures and Sum-Product Networks. (2026). *Stats*, 9(4), Artículo 84. <https://doi.org/10.3390/stats9040084>
- [6]. Hinton, G. E., Osindero, S., Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), 1527–1554. <https://doi.org/10.1162/neco.2006.18.7.1527>
- [7]. Mocanu, D. C., Mocanu, E., Nguyen, P. H., Gibescu, M., Liotta, A. (2016). A topological insight into restricted Boltzmann machines. *Machine Learning*, 104(2–3), 243–270. <https://doi.org/10.1007/s10994-016-5570-z>
- [8]. Fischer, A., Igel, C. (2014). Training restricted Boltzmann machines: An introduction. University of Copenhagen Research Portal.